Performance Testing

Date	1 July, 2026
Team ID	
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	**Details**
Testing Tool Used	Apache JMeter, Postman
Type of Testing	Load Testing, Stress Testing
Target Module	Flask Prediction API (/predict), User Input Form
Test Environment	Local System (Windows 11, Python 3.11, Flask)
Test Date	1 July, 2026

Step 2: Test Scenarios

S.No	**Test Scenario / Description**	**No. of virtual Users**	**Duration (sec)**	**Expected Outcome**
1	Single user submits credit card application	1	30	Prediction generated successfully within 2 seconds
2	Continuous prediction requests	100	120	Stable response time with no crashes
3	Peak load testing	200	180	Application remains responsive and maintains acceptable performance

Step 3: Performance Test Results

S.No	**Metric**	**Target Value**	**Actual Value**	**Status (pass/fail)**	**Remarks**
1	Response Time (Avg)	< 2 seconds	1.28 sec	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	3.63 sec	Pass	Within acceptable limit
3	Throughput (Req/sec)	>40 req/sec	52 req/sec	Pass	Good request handling capacity
4	Error Rate	<1%	0%	Pass	No request failures
5	CPU Utilization	<80%	61%	Pass	Efficient CPU usage

6	Memory Utilization	<80%	57%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction system successfully processed concurrent user requests.
- Average prediction response time remained below two seconds.
- No failed prediction requests were observed during testing.
- Flask application remained stable under moderate and heavy workloads.
- Machine learning model delivered accurate predictions without performance degradation.

Bottlenecks Identified

- Minor increase in response time when more than 200 concurrent users accessed the application.
- Initial model loading caused a slight delay during the first prediction request.

Optimization Steps Taken

- Optimized Flask routing and request handling.
- Reduced unnecessary preprocessing operations.
- Loaded the trained ML model once during application startup.
- Enabled efficient NumPy operations for faster predictions.
- Improved input validation to reduce processing overhead.